Computer Vision & Python: Tools Survival Guide

CV and ML Development

Python Jupyter conda git DVC VSCode DVC Studio CML MLFlow Spark …

Mathias Kölsch, January 2026

Mathias' General Recommendations

- Use Anaconda
- Use conda
  - Use Python virtual environments via "conda create" not penv
  - Install packages with conda if available, not pip
- Use JupyterLab Server (it's a superset of Jupyter Notebook server)
- VS Code with Jupyter is more powerful (terminal integration, git, remote connections, 1000s of extensions) but …
- JupyterLab is easier to get started if you're not familiar with an IDE

VS Code is Python-aware and conda-aware

- Language support and code formatting
- Debugger can run Python code
- Python interpreters (kernels) can be chosen
- A specific Python virtual env can be selected: virtual env determines library versions and interpreter version
- Conda-managed environments can also be selected
- Command Palette (⇧⌘P) shift-command-P, then…
  - → Python: Select Interpreter (this also selects the environment)
  - → Python: Create Environment
  - → settings.json: change user settings

Which Version?

```
!which python
import sys
print("Python version:", sys.version)
print("Library paths:")
for path in sys.path:
    print(path)
import os
!conda info
print (os.environ['CONDA_DEFAULT_ENV'])
print(os.environ["CONDA_PREFIX"])

import cv2
print(f"Library path for cv2:", cv2.__file__)
print("OpenCV Version: ", cv2.__version__)
import matplotlib.cbook as cbook
imgpath = cbook.get_sample_data('grace_hopper.jpg',asfileobj=False)
img = cv2.imread(imgpath)
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
import matplotlib.pyplot as plt
plt.imshow(img)

import tensorflow as tf
print("TensorFlow version: ", tf.__version__)
print("Keras version: ", tf.keras.__version__)
```

Fix your environment in a Terminal

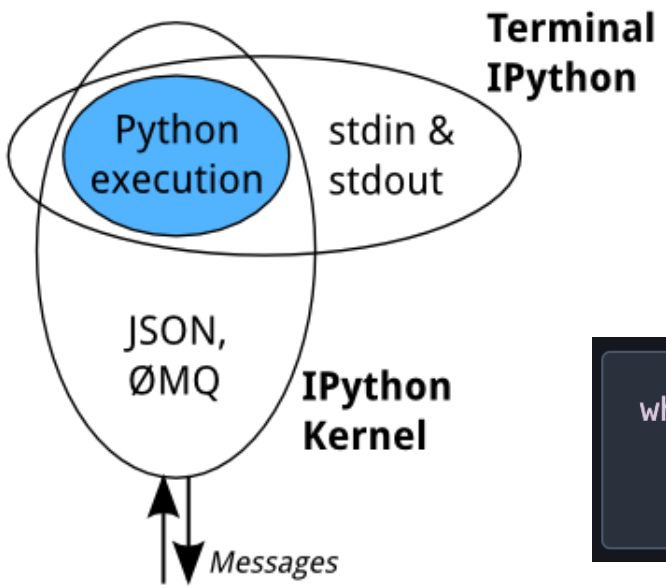
```
conda env list                    # or conda info –envs # list envs
conda create --name cs4330
# conda create --name cs4330python312 python=3.12
python -m ipykernel install --user --name=cs4330
conda activate cs4330
conda install opencv pillow
conda install conda-forge::hyperopt conda-forge::pyspark
conda install mlflow
conda list                        # show libs & versions
conda clean --all                 # maintenance, in case of errors
conda install nb_conda            # for switching kernels
```

register the new environment in Jupyter, so it shows up as selectable kernel

Note double-colon

Jupyter = iPython and ipy kernel

https://docs.jupyter.org/en/latest/projects/architecture/content-architecture.html

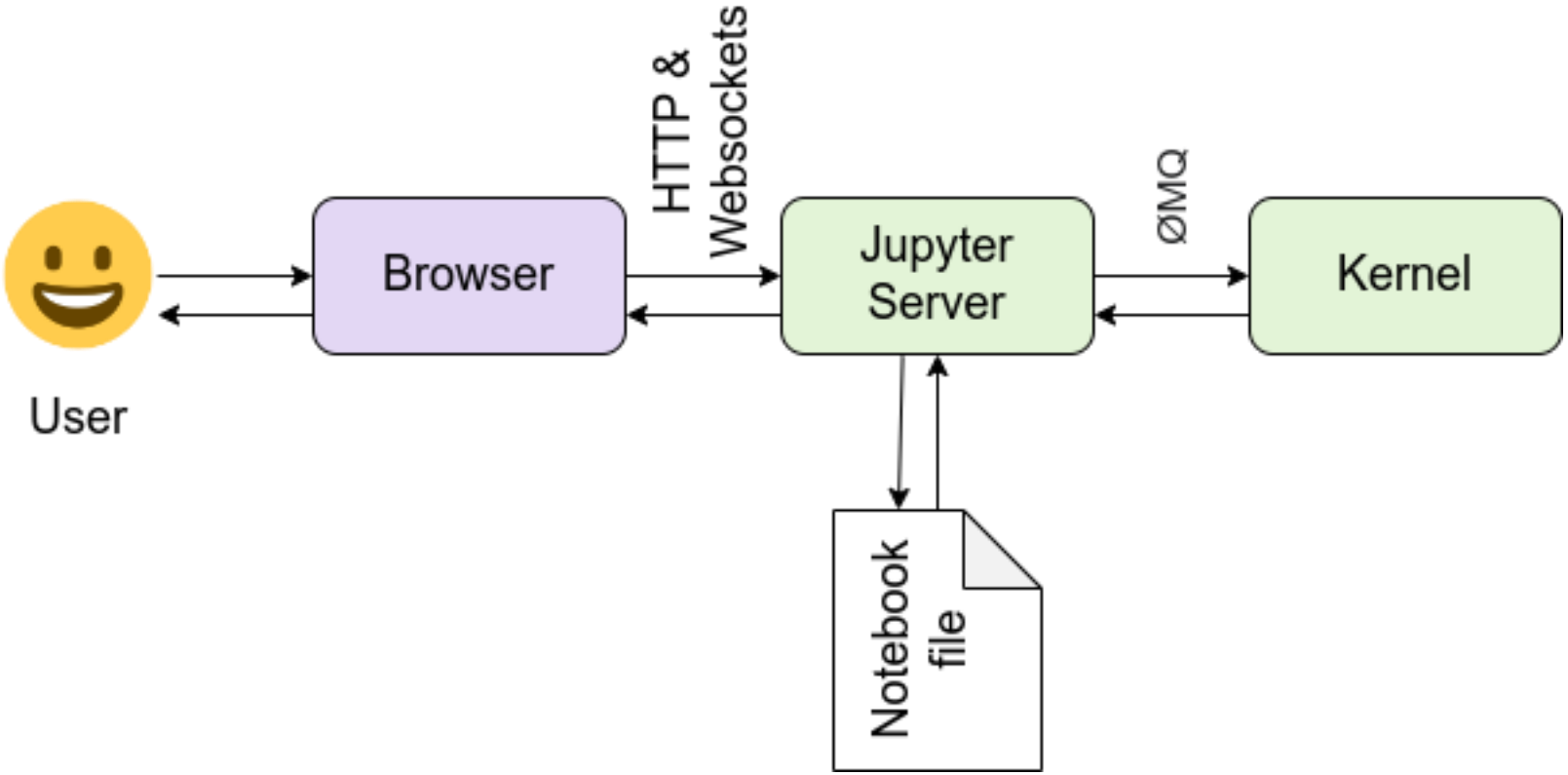


Notebook

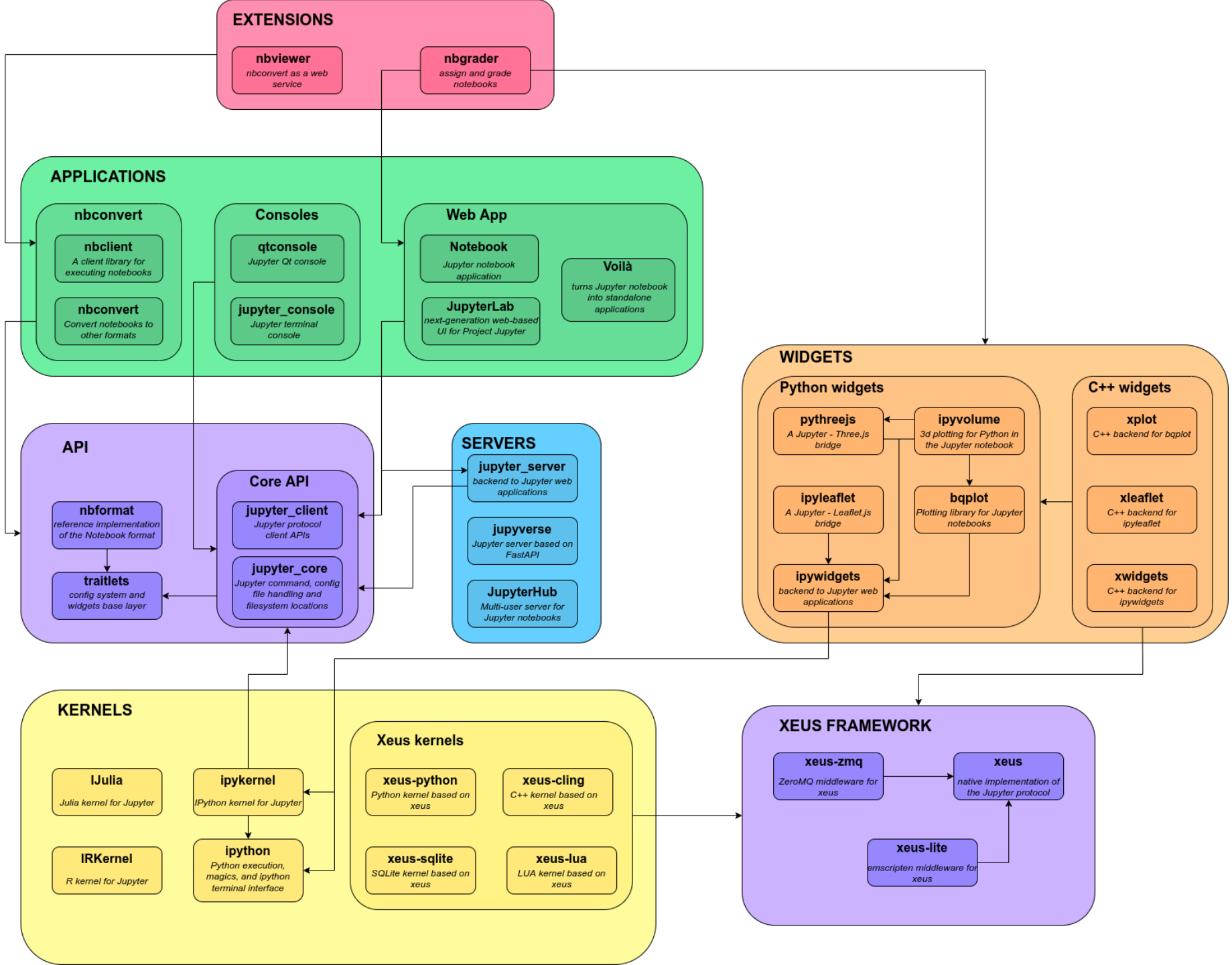
```
(base) mathiaskolsch@mac00093-csclpp ~ % ipython
Python 3.11.5 (main, Sep 11 2023, 08:19:27) [Clang 14.0.6 ]
Type 'copyright', 'credits' or 'license' for more information
IPython 8.15.0 -- An enhanced Interactive Python. Type '?' for help.

In [1]: import son
  abc
  aem
  aeosa
  aifc
  aiobotocore
  aiofiles
```

```
while True:
    code = input(">>> ")
    exec(code)
```

Jupyter server: single-user web app



Behind the scenes: Jupyter modules



JupyterLab: the “better” Jupyter notebook

- JupyterLab is a more flexible successor to the Notebook, and another interface to the ipykernel.
  - Can run multiple documents, show files, etc.
  - Integrates terminal, plugin viewers etc.
  - More: see this SaturnCloud explanation on the differences

- Attempts at enabling collaboration in notebooks:
  - https://jupyterlab-realtime-collaboration.readthedocs.io/en/latest/
  - Databricks has a robust solution, AWS EMR Studio is so-so

JupyterHub: hosts notebooks and JupyterLabs

- “JupyterHub is a multi-user Hub that spawns, manages, and proxies multiple instances of the single-user Jupyter notebook server.”

- Reuses users on the Hub’s host OS, or via authentication plugins
- Notebook storage is by default on the host OS
- Collaboration?

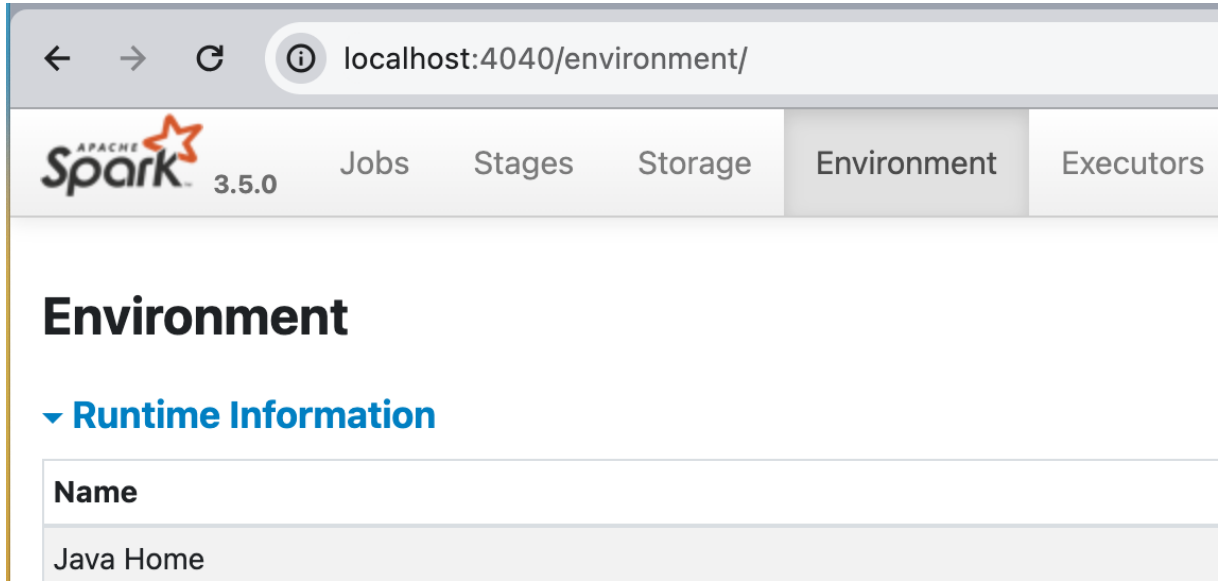
Hosted Notebooks and Hubs

- Host your own with Anaconda
- Google Colab (fully managed)
- AWS EMR (Elastic MapReduce)
  - EMR Studio (fully managed)
  - EMR Studio Workspace = EMR Notebook (on AWS console, S3 for notebooks storage, connects to Spark)
  - EMR lets you host a custom JupyterHub

Your own Spark Cluster

```
# Spark needs Java to run
conda install -c conda-forge pyspark
# or if that doesn't work: pip install pyspark
java –version
pyspark
# then http://localhost:4040

# Ray library for parallelization:
# https://docs.ray.io/en/latest/index.html
```

needs to be running



Hyperparameter Optimization

- Popular frameworks:
  - Hyperopt: https://github.com/hyperopt/hyperopt
  - Ray Tune: https://docs.ray.io/en/latest/tune/index.html
  - Optuna: https://optuna.org
- https://docs.databricks.com/en/machine-learning/automl-hyperparam-tuning/hyperopt-spark-mlflow-integration.html
- See local hyperopt.ipynb

DevOps and CI/CD with Data and Models

- DVC: Data Version Control so you can track changes to your datasets
- MLFlow: experiment tracking, model versioning, deployment
  http://mlflow.org
- CML: Continuous Machine Learning - automation of ML workflows
  https://cml.dev/

Dask

Similar to Spark, there's also Dask https://docs.dask.org/en/stable/

- Dask Distributed for job scheduling
- Dask Dataframes for larger dataframes than what a single node can hold

Sample Project: Tamil characters

- https://www.kaggle.com/competitions/tamil-hwcr/overview